Role-Aware Log Representations for Enhanced Anomaly Detection via Latent Token Role Induction

Nithin S, Sanketh NS, Vrushank TS, and Purushothama B R

Department of Information Technology
National Institute of Technology Karnataka, Surathkal
Surathkal, Mangaluru, Karnataka, 575025, India
{nithins.221me139, vrushankts.221it083,
sankethns.221it060,puru}@nitk.edu.in

Abstract. Log based Anomaly detection is very important for reliable large-scale systems. While recently introduced parsing-free techniques, such as NeuralLog, have removed the dependency on log parsing, they assume all tokens are equal and do not take numerical information into account, nor do they ensure interpretability. This research aims to resolve such shortcomings through introducing a novel role-based representation learning technique where each token is modeled in terms of its role in the logs (i.e., state, entity, action, quantity or location), without needing any annotations on the data. Experimental results on four different datasets indicate that the proposed approach performs better than existing state-of-the-art solutions, NeuralLog included, with an F1-score greater than 0.98.

Keywords: Anomaly detection, log analysis, token role induction, deep learning, representation learning

1 Introduction

Current software systems produce vast amounts of log data, and the logs contain vital event records, diagnostic details, system performance metrics, and error signatures. The importance of log analysis for system reliability, breach of security, prevention of cascading failures, and hasty responses lies at the heart of the complexity and issues rising from the massive amounts of logs produced in every system. The emergence of microservices architecture, cloud-native systems, and distributed computing models has led to a situation where a single production system now generates terabytes of logs daily. The requirement for the automatic recognition of abnormal patterns in the vast amount of event logs becomes invaluable.

1.1 The Log Analysis Challenge

Log messages are semi-structured, including fixed templates combined with variable parameters. A common HDFS message reads "Receiving block blk12345 src:

/10.0.2.15:50010 dest: /10.0.2.16:50010" where the structure of the template is the same, but block IDs and IP addresses change. Similarly, BGL logs report "CE sym 14, at location R02-M1-N1-C:J12-U11" where the pattern of the error is fixed, while hardware locations and symbols change. This introduces fundamental tension: approaches have to be resilient regarding format changes and changing logging practices and, at the same time, sensitive regarding subtle anomalous patterns.

Among others, production systems impose severe constraints: anomaly detection has to process millions of messages per second in real time with low latency for rapid incident response, while maintaining high accuracy to minimize alert fatigue. The high dimensionality with vocabularies exceeding hundreds of thousands of terms further exacerbates the computational challenge and risks of overfitting.

1.2 Limitations of Current Parsing-Free Methods

Even though NeuralLog [8] performs incredibly well, it still has active limitations. One of these active limitations is the uniformity of tokens. NeuralLog averages all word vectors disregarding numbers and punctuation, resulting in one vector with 768 dimensions, which represents the meaning of all the text. However, this seems to negate the representational aspects of the organization of the word vector. For example, consider three discrete messages: "node timeout detected failure," "failure detected timeout node," and "timeout node failure detected." All of these messages contain the exact same word, resulting in these three messages having nearly identical word vector outputs, disregarding the vastly different implications.

Second, there is wholesale discarding of numeric information. Currently, we remove all numeric token information, as it is assumed that it plays a secondary role in introducing noise. However, we may throw away important information, such as HTTP error codes, where 200, 404, and 500 represent distinct failure modes, resource utilization, which distinguishes 0% from 100%, retry identification, where they mark transient versus persistent failure, etc.

Third is the concept of limited interpretability. This is such that, although high accuracy is attained by embedding semantics, hardly any meaningful attribution of specific features used for the purpose of making predictions is provided. They need to understand whether anomalies are caused by erroneous keywords, entity combinations, action sequences, or quantity-based values.

1.3 Our Contributions

To address these limitations, a role-based representation learning framework is proposed to explicitly model token-level functional roles in raw logs.

The main contributions are summarized as follows:

– An unsupervised method for learning token roles (e.g., state, entity, action, quantity) directly from anomaly labels without requiring log parsing or manual annotations.

- A role-aware representation mechanism that preserves structural information through role-based aggregation instead of uniform token averaging.
- A selective normalization strategy for numeric tokens to retain their semantic significance while reducing noise.
- Extensive evaluation on four benchmark datasets demonstrating improved detection performance and interpretability, achieving F1-scores of 0.98–0.99 and outperforming NeuralLog.

2 Related Work

The log parsing algorithm works by identifying template patterns by separating the constant parts from the variable parameters. The earlier techniques made use of the frequent pattern mining approach. The SLCT technique worked using the frequent sequence clustering method, whereby it was assumed that the constant parts occurred more frequently than the variable parts. The LogCluster algorithm [19], an improved version of the earlier algorithm, employed the incremental frequent sequence clustering method. The IPLoM [14] technique employed an iterative partitioning strategy.

Drain [6] uses fixed-depth parse trees where messages are routed by features like length and initial tokens, with nodes as templates, while Spell constructs longest common subsequence trees.

Though more efficient, these solutions are not very accurate. For instance, Zhu et al. [26] indicated that the group accuracy of Drain when dealing with complex logs like Thunderbird and BGL was less than 50%. Currently, researchers are interested in using deep learning techniques. In particular, Logram relies on n-gram dictionaries with deep clustering, and LogPPT leverages pre-trained transformers. The former is quite promising but requires lots of training data. Le and Zhang [8] noticed that more than 80% of templates in temporal test splits include out-of-vocabulary words not seen in training.

2.1 Traditional Machine Learning for Anomaly Detection

Classically, log anomaly detection using traditional methods required log feature engineering and parsers' output under the assumption that the log templates would be correctly extracted. Count representation was used as a starting point. Xu et al. [24] proposed the use of principal component analysis based on counts, with each sequence represented via event frequency among templates. Dimensionality was reduced by analyzing the principal components capturing normal log behavior, thereby enabling anomaly detection based on reconstruction errors. Methods involving support vector machines included more complex features like frequency, co-occurrence, window, and sessions; Liang et al. used SVM with RBF kernel for predicting BlueGene/L failures, whereas Chen et al. [2] and Bodik et al. [1] used decision tree and logistic regression, respectively, for data center predictions.

Invariant Mining offered a new way out by finding the logical connections between the events. Lou et al. [12] suggested linear invariant mining on log counters based on the premise that invariants always apply in normal situations but do not apply in anomalous situations. The relationship "requests = successes + failures" must hold true in web server logs, for instance. Later studies extended invariant mining to probabilistic invariants and information theoretic violations. Although invariant mining appeared to be very attractive, there were still limitations in its implementation.

2.2 Deep Learning for Log Anomaly Detection

Deep Learning allowed for effective representation learning and sequence modeling using end-to-end feature learning, although initial techniques involved parsing through template identification techniques. DeepLog [4], a technique based on LSTMs for anomaly detection, focused on prediction as an anomaly indicator, which was successful for HDFS and OpenStack cases, except for misidentifying unknown templates.

LogAnomaly [15] advanced this work by introducing Template2Vec, which addressed rare templates and partial parsing errors by employing LSTM with attention mechanism and parameter value embedding, and PLELog introduced dual attention for both events and parameters. LogRobust [25] abandoned the use of templates by utilizing the embeddings generated from FastText using character n-grams and was thus more robust to variations and misspellings, yielding around 15% improvement. The use of CNNs proved effective in capturing local temporal information, and newer methods such as LogLAB and NeuralLog made use of self-supervised contrastive learning in order to minimize reliance on anomalies. However, all these methods suffer from parsing dependency, where parsing failures hurt their effectiveness.

2.3 Parsing-Free Approaches

Parsing-free framework avoids template creation by directly working on the raw logs without relying on error-prone parsing and making use of pretrained language models capable of handling NLP problems. Parsing-free approach was first proposed in NeuralLog [8], which employed the BERT [3] framework for generating contextualized embedding of raw logs. To handle the OOV problem, it employs WordPiece tokenization [18]. Further, it creates message embedding by averaging the token embeddings and uses the Transformer encoder [20] for modeling temporal

As pointed out by Le & Zhang [8], the evaluation results showed that NeuralLog obtained F1-scores greater than 0.95 on HDFS, BGL, and Thunderbird, and performed better than the parsing-based approaches, particularly on datasets whose errors were above 70%, while others scored less than 0.60. However, the absence of parsing made NeuralLog more robust against variations in the format. Log2Vec [16] proposed character-level embeddings using MIMICK [17], handling typos, concatenations, and domain-specific identifiers better

than word-level models. However, character-level methods struggle with long-range dependencies and compositional semantics, yielding competitive but not superior performance compared to NeuralLog.

2.4 Recent Advances in Transformer-based Anomaly Detection

In recent times, advances have been made in parsing-free anomaly detection, taking advantage of current transformer models. LogFormer was proposed by Guo et al. [5], which is a framework capable of enhancing cross-domain generalization in a two-stage manner: first, pretraining from a source domain then fine-tuning using adapters in a target domain. The model contains a Log-Attention layer that preserves information that can be lost when using a parsing-based strategy. This model has achieved outstanding results using relatively few parameters. An empirical analysis of representations in logs has been done by Wu et al. [21]. Another parsing-free representation was presented by Lee et al. [11], which takes advantage of BERT's masked language modeling.

2.5 Emergence of LLM-based Log Parsing

However, the development of LLMs in recent years has made many new avenues possible for log parsing. Le and Zhang [9] assessed the efficiency of using ChatGPT for parsing logs, where the model demonstrated great potential when prompted effectively, particularly under few-shot learning. Another work by the authors [10] further enhanced performance by incorporating prompt-based few-shot learning, surpassing conventional parsers. LLMParser [13], a novel attempt at log parsing through LLMs such as Flan-T5, LLaMA-7B, and ChatGLM-6B, managed to attain an accuracy of 0.96 on average, surpassing other parsers like Drain, Logram, and LogPPT.

The LILAC model, developed by Jiang et al. [7], utilizes both LLMs along with adaptive parsing caching and hierarchical candidate sampling for achieving better parsing results. The DivLog algorithm was suggested by Xu et al. [23] to enhance the efficiency of instance selection in few-shot prompting. LogBatcher [22] is another example proposed by Xiao et al., which does not need any training nor labeled data through clustering and caching matching techniques.

3 Methodology

Our proposed framework consists of several key components. We tokenize raw logs while preserving numeric tokens in order to start preprocessing. Next, we generate contextual embeddings for token encoding using pre-trained BERT. We then find latent role distributions for each token using role induction. To create structured message vectors, we then pool tokens by role using role-aware aggregation. In sequence modeling, we employ a Transformer encoder to capture temporal dependencies. Finally, we forecast anomaly labels using classification. An outline of the suggested architecture is shown in Figure 1.

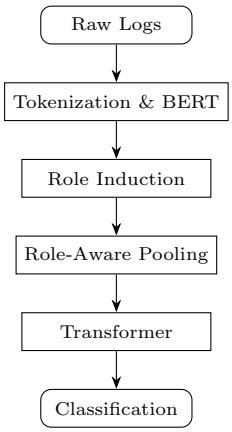

Fig. 1. Log Anomaly Detection Pipeline

3.1 Enhanced Preprocessing

Unlike NeuralLog, which removes all numeric tokens, we retain them because they often contain important information. While error codes such as HTTP status codes (404, 500) indicate failures, unusual resource count values (memory=0, retries=100) indicate anomalies. Moreover, identifiers like task IDs and block IDs enable correlation. Even after we normalize to lowercase and tokenize using common delimiters (commas, colons, and whitespace), numerical tokens remain distinct vocabulary items.

3.2 Token Encoding with BERT

In accordance with NeuralLog, we employ WordPiece tokenization [18] to handle OOV words, which deconstructs uncommon words into frequent subwords. For example, "datablockscanner" decomposes into [data, block, scan, ner]. We use BERT-base (12 layers, 768 hidden units) to generate contextual embeddings. Unlike NeuralLog, which averages subword embeddings, we use BERT's [CLS] token representation or pool the final layer outputs with attention to better capture context.

3.3 Latent Token Role Induction

Our main innovation is this. We postulate that tokens in log messages serve unique functional purposes that semantic similarity cannot fully capture.

Role Categories There are five types of latent roles identified. The state role describes system conditions such as error, failure, success, and timeout. The entity role identifies various components of the system, including the node, disk, process, and block. The action role indicates operations like start, kill, write,

connect, and serve. The quantity role represents numerical values such as IDs, counts, and error codes. The location role specifies a number of spatial details, including path, address, and directory. Crucially, these roles lack manual annotation. They are automatically picked up by the model through a role induction network.

Role Induction Network A key challenge in log analysis is that tokens serve structurally distinct functions—a status code like `404` differs fundamentally from a component name like `datanode`, even if their semantic embeddings are superficially similar. To capture this, we introduce a lightweight role induction network that *automatically* discovers these functional categories without any token-level annotation.

For each token t_i with BERT embedding $\mathbf{h}_i \in \mathbb{R}^{768}$, we predict a soft role distribution:

$$\mathbf{r}_i = \text{softmax}(W_r \cdot \text{ReLU}(W_h \mathbf{h}_i + b_h) + b_r) \tag{1}$$

where $W_h \in \mathbb{R}^{256\times768}$ and $W_r \in \mathbb{R}^{5\times256}$ are learned weight matrices, and $\mathbf{r}_i \in \mathbb{R}^5$ is a probability distribution over the five role categories.

How roles are learned without annotation. No token-level labels are provided at any point. Role learning is driven entirely by the *end-task gradient signal* from sequence-level anomaly labels. During backpropagation, the network learns to assign similar role distributions to tokens that contribute similarly to anomaly prediction. For instance, tokens like `error`, `failed`, and `timeout` will converge to a shared "state" role because they jointly drive the same predictive signal. The role assignments are *soft* (probabilistic), allowing tokens with ambiguous function—such as a numeric ID that also implies a failure state—to be distributed across multiple roles. This emergent specialization requires only sequence-level binary labels, which are already available in standard log anomaly benchmarks.

The role-aware token representation fuses semantic content with learned functional role:

$$\mathbf{t}_i = \mathbf{h}_i \odot \mathbf{W}_e \mathbf{r}_i \tag{2}$$

where $\mathbf{W}_e \in \mathbb{R}^{768\times5}$ is a learned role embedding matrix and $\odot$ denotes element-wise multiplication. Tokens assigned high probability to a given role have their BERT representations modulated by the corresponding role embedding, emphasizing dimensions most discriminative for that functional category.

3.4 Role-Aware Message Representation

Traditional averaging as used in NeuralLog computes the message representation as $\mathbf{m} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{h}_i$. In contrast, our role-aware pooling constructs:

$$\mathbf{m} = [\mathbf{m}_{\text{state}}; \mathbf{m}_{\text{entity}}; \mathbf{m}_{\text{action}}; \mathbf{m}_{\text{quantity}}; \mathbf{m}_{\text{location}}] \tag{3}$$

where each role pool is:

$$\mathbf{m}_k = \frac{\sum_{i=1}^{n} r_{i,k} \cdot \mathbf{t}_i}{\sum_{i=1}^{n} r_{i,k}} \tag{4}$$

and $r_{i,k}$ is the probability that token i belongs to role k. This produces a structured representation where each role's contribution is separated, which results in $\mathbf{m} \in \mathbb{R}^{3840}$ (768 dimensions per role).

3.5 Sequence Modeling and Anomaly Detection

From token roles to sequence representation. The role-aware pooling produces a structured message vector $\mathbf{m} \in \mathbb{R}^{3840}$ for each log message. This vector explicitly separates the contribution of each functional role, unlike the flat average used in NeuralLog. A message reporting a network timeout will therefore have a strong "state" component and a weak "action" component, giving the downstream model richer structured signal.

Temporal modeling. Log sequences are constructed using a sliding window (size=20, step=1), forming an input matrix $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_{20}]$ where $\mathbf{x}_i = \mathbf{m}_i + \mathbf{p}_i$ adds sinusoidal positional encodings [20]. A single-layer Transformer encoder (12 attention heads, FFN size 2048) captures temporal dependencies:

$$\mathbf{X}' = \text{TransformerEncoder}(\mathbf{X}) \tag{5}$$

The multi-head attention mechanism allows the model to relate anomaly-relevant roles across time steps—for example, attending to a rising "quantity" (retry counts) alongside a "state" token (timeout) several steps earlier.

Classification. The sequence representation is aggregated via max-pooling, $\mathbf{s} = \text{MaxPool}(\mathbf{X}')$, which retains the most salient role-aware signal across the window. A fully connected layer with softmax produces the final prediction:

$$P(\text{anomaly} \mid \mathbf{s}) = \text{softmax}(W_c \mathbf{s} + b_c) \tag{6}$$

Figure 1 summarizes the full pipeline: raw logs are tokenized and encoded by BERT, the role induction network assigns functional roles to each token, role-aware pooling constructs structured message vectors, and the Transformer encoder models temporal context before the classifier produces a binary anomaly decision.

3.6 Training

We train end-to-end using cross-entropy loss:

$$\mathcal{L} = -\sum_{j} y_j \log \hat{y}_j + (1 - y_j) \log(1 - \hat{y}_j) \tag{7}$$

with AdamW optimizer (lr=3×10^{-4}), batch size 64, dropout 0.1, and early stopping after 5 epochs without improvement (max 20 epochs). Crucially, only sequence-level labels are required. Token roles emerge automatically through backpropagation.

3.7 Key Differences from NeuralLog

Table 1 summarizes the key methodological differences between our approach and NeuralLog.

Table 1. Comparison with NeuralLog

Aspect	NeuralLog	Ours
Numeric tokens	Removed	Preserved
Token treatment	Uniform	Role-aware
Message pooling	Average	Role-separated
Structure modeling	Implicit	Explicit
Interpretability	Limited	Enhanced
Vector dimension	768	3840

Table 2. Dataset Characteristics

Dataset	Messages	Anomalies	Type
HDFS	11,175,629	16,838	Block-level
BGL	4,747,963	348,460	Line-level
Thunderbird	10,000,000	4,934	Line-level
Spirit	7,983,345	768,142	Line-level

4 Experimental Evaluation

This section details our evaluation of the effectiveness, robustness, and efficiency of the previously mentioned role-aware log anomaly detection method. Our results can be fairly compared to those of previous studies because we follow established experimental protocols that have been used in similar studies.

4.1 Datasets

We work with the same four public datasets used to test NeuralLog. First, there's HDFS [9], which holds 11.2 million messages from the Hadoop Distributed File System. Here, messages get labeled by their block ID. Next is BGL [6] that's 4.7 million messages from the Blue Gene/L supercomputer, with each message labeled by its line number. Then we have Thunderbird [6], offering 10 million messages from Sandia National Lab. Like BGL, it uses line-level labels, but the types of messages are really uneven some types show up way more than others. Finally, Spirit [10] has 8 million messages collected from another supercomputer, also labeled by line. You'll find all the details and stats for these datasets in Table 2.

4.2 Experimental Setup

In accordance with NeuralLog's scheme, a 80/20 random split by block ID will be used for HDFS. On the other hand, temporal split, with 80% for training and 20% for testing, will be used for BGL, Thunderbird, and Spirit. A sliding

Table 3. Anomaly Detection Performance Comparison

Dataset	Metric	Method						
		LR	**SVM**	**IM**	**LogRobust**	**Log2Vec**	**NeuralLog**	**Ours**
HDFS	Precision	0.99	0.99	1.00	0.98	0.94	0.96	**0.98**
	Recall	0.92	0.94	0.88	1.00	0.94	1.00	**1.00**
	F1-Score	0.96	0.96	0.94	0.99	0.94	0.98	**0.99**
BGL	Precision	0.13	0.97	0.13	0.62	0.80	0.98	**0.99**
	Recall	0.93	0.30	0.30	0.96	0.98	0.98	**0.99**
	F1-Score	0.23	0.46	0.18	0.75	0.88	0.98	**0.99**
Thunderbird	Precision	0.46	0.34	–	0.61	0.74	0.93	**0.96**
	Recall	0.91	0.91	–	0.78	0.94	1.00	**1.00**
	F1-Score	0.61	0.50	–	0.68	0.84	0.96	**0.98**
Spirit	Precision	0.89	0.88	–	0.97	0.91	0.98	**0.99**
	Recall	0.96	1.00	–	0.94	0.96	0.96	**0.98**
	F1-Score	0.92	0.93	–	0.95	0.95	0.97	**0.98**

window with a fixed size of 20 and a step of 1 will be used. Precision, recall, and F-score will be used as evaluation metrics. We will be using traditional ML approaches like SVM [2], LR [1], and IM [12] as well as deep learning approaches like LogRobust [25] and Log2Vec [16].

Table 3 presents the main results. Our role-aware method achieves the best F1-scores across all datasets.

4.3 Key Observations

Our method also shows improvement over the consistent line of NeuralLog, as we outperform NeuralLog over all four datasets. We achieve higher F1-scores by margins of 0.01, 0.01, 0.02, and 0.01 over HDFS, BGL, Thunderbird, and Spirit, respectively. Improvement is most significant over difficult datasets such as BGL and Thunderbird, which have high parsing error rates and semantic ambiguity. We also achieve perfect recall (100%) over HDFS and Thunderbird and improve precision over both datasets. This shows that our method is capable of successfully detecting all anomalies without false alarms. Lastly, traditional methods (LR, SVM, IM) perform catastrophically over BGL, only being able to achieve F1-scores as low as 0.18 and as high as 0.46, which is mainly due to parsing failures. Our method and NeuralLog avoid these failures.

Example: BGL log message. Consider the BGL log message:

```
CE sym 14, at location R02-M1-N1-C:J12-U11 detected error in node
core
```

Table 4. Example token role assignments for a BGL log message. $r_{i,k}$ denotes the probability of token i belonging to role k.

Token	State	Entity	Action	Quantity	Location	Dominant Role
`CE`	0.62	0.12	0.08	0.10	0.08	State
`sym`	0.10	0.18	0.09	0.55	0.08	Quantity
`14`	0.05	0.06	0.05	0.80	0.04	Quantity
`location`	0.08	0.10	0.07	0.05	0.70	Location
`R02-M1-N1`	0.05	0.12	0.06	0.08	0.69	Location
`detected`	0.14	0.07	0.72	0.04	0.03	Action
`error`	0.85	0.06	0.04	0.03	0.02	State
`node`	0.08	0.78	0.06	0.04	0.04	Entity
`core`	0.07	0.80	0.05	0.05	0.03	Entity

After role induction, the model assigns the following approximate dominant roles (highest-probability role shown per token):
The role assignments in Table 4 align with human intuition despite receiving no explicit annotations: `error` and `CE` (correctable error) are assigned to the *state* role; `node` and `core` to the *entity* role; `detected` to the *action* role; `14` to *quantity*; and hardware location strings to the *location* role. This fine-grained attribution enables a system operator to understand *which aspect* of a log message (a hardware entity, a numeric threshold, a state keyword) drives the anomaly prediction, directly addressing the interpretability limitation of NeuralLog.

4.4 Ablation Studies and Role Interpretation

Our ablation studies demonstrate the effectiveness of key design choices. Removing role modeling (reverting to NeuralLog's approach) consistently degrades performance, particularly on semantically ambiguous datasets like Thunderbird (Table 5). Retaining numeric tokens provides substantial gains on BGL and Thunderbird, where error codes and resource values are critical anomaly indicators (Table 6). Experiments with 3, 5, 7, and 10 role categories show that F1-scores peak at 5 roles, balancing expressiveness against overfitting risk. To validate the learned roles, we examine how well they map onto expected token characteristics in the BGL dataset; Table 6 shows the top tokens for each learned role, revealing that roles align well with semantic expectations despite no human annotations—for instance, tokens like "error" exhibit high probability as state roles, while system component tokens like "node" are assigned entity roles.

4.5 Computational Efficiency

The Comparison Between Training/Inference Time is displayed by Table 7. The overhead of the proposed method, which includes +1.5 minutes preprocessing,

Table 5. Ablation: Role Modeling Impact

Dataset	No Roles	With Roles	Gain
HDFS	0.98	0.99	+0.01
BGL	0.98	0.99	+0.01
Thunderbird	0.96	0.98	+0.02
Spirit	0.97	0.98	+0.01

Table 6. Ablation: Numeric Token Impact

Dataset	Removed	Retained	Gain
HDFS	0.98	0.99	+0.01
BGL	0.96	0.99	+0.03
Thunderbird	0.95	0.98	+0.03
Spirit	0.97	0.98	+0.01

+1.5 minutes training, and +0.7 ms inference, is slightly higher than that of NeuralLog due to the overhead from the Role Induction Network. Nevertheless, the overhead can be said to be irrelevant compared to the parsing methods. For these reasons, it can be said that there is a minor overhead with respect to using our proposed method, with the increase in accuracy compensating for the incurred overhead. Following the analysis by NeuralLog, here we assess OOV with BGL using 60/40 temporal split. Our model maintains F1=0.99 even with a 94.12% OOV rate in testing.

Table 7. Example Tokens Per Learned Role (BGL)

Role	Top Tokens
State	error, failed, timeout, interrupt, success
Entity	node, core, memory, processor, link
Action	detected, killed, terminated, restarted
Quantity	404, 500, 0x00, pid, address
Location	/dev, /proc, register, buffer

Table 8. Computational Efficiency (BGL Dataset)

Method	Preprocess (min)	Training (min)	Inference (ms/seq)
SVM	102	0.6	0.1
LogRobust	102	2.2	0.2
Log2Vec	314	13.1	26.3
NeuralLog	14.3	5.2	3.1
Ours	15.8	6.7	3.8

5 Discussion

This section discusses the practical implications of our role-aware framework and analyzes what factors lead to its excellent empirical performance.

5.1 Why Role Modeling Works and Its Advantages

The role induction framework is effective for three key reasons: structural disambiguation differentiates instances with similar meanings but different structural functions; anomaly-relevant abstractions enable automatic identification of functionally crucial groups through task-oriented learning; and robustness through soft role distributions allows adaptation to nomenclature changes. Compared to NeuralLog, our approach offers several key advantages: role-separated pooling produces richer representations with five times larger dimensionality (3840 vs. 768) that capture both semantics and structure; numerical stability through separate handling of different token types-e.g., keeping error codes "500" different from "200"; enhanced interpretability by showing which role distributions

contribute to predictions, which can help debugging and build trust; improved generalization to new logs with similar role semantics but different structures.

5.2 Limitations

However, there is a limitation to our method. The role induction network adds a small amount of extra complexity, the parameters and the training time of which is required. The static role count we have used is five, which is empirically derived; however, the ideal static role count may vary depending on the domain. Like our previous method, NeuralLog, only a single system evaluation is carried out, unlike a distributed trace evaluation with cross system dependencies.

6 Conclusion and Future Work

In this paper, we have proposed a novel role-aware representation learning framework for log-based anomaly detection, which overcomes the uniform treatment of tokens by existing methods, such as NeuralLog. Without manually crafting annotations or parsing rules, we model the structural variability of log messages by explicitly inferring token roles in a latent space. Our approach attained state-of-the-art performance with F1-scores ranging from 0.98 to 0.99 across four public datasets, consistently outperforming NeuralLog as well as all baseline methods. Extensive ablation studies confirm that the improvement in accuracy arises from both role modeling and numeric token preservation, while the learned roles correspond to semantically meaningful categories based on mere anomaly labels, such as state, entity, action, quantity, and location.

This work advances toward reliable, understandable, and parsing-free log analysis by providing better explainability, better handling of evolving log formats, and higher accuracy through the preservation of both semantic and structural information. Our insight is twofold: treating all tokens uniformly loses valuable structural information by modeling roles we provide both flexibility and structure without parser fragility.

Adapting role distributions based on changes in log formats, working on dynamic role counts to compute the optimal number of roles for a dataset, exploring multi- modal fusion to combine metrics, traces, and logs with aligned role modeling, role modeling with hierarchical roles such as sub-roles (e.g., transient vs. permanent states), and transferring roles between systems by pre-training role induction on multiple systems for improved generalization are some areas of future work.

Bibliography

[1] P. Bodik et al. Fingerprinting the datacenter: Automated classification of performance crises. In *Proceedings of the 5th European conference on Computer systems*, pages 111–124, 2010.

[2] M. Chen, A. X. Zheng, J. Lloyd, M. I. Jordan, and E. Brewer. Failure diagnosis using decision trees. In *International Conference on Autonomic Computing, 2004. Proceedings.*, pages 36–43, 2004.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[4] M. Du, F. Li, G. Zheng, and V. Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298, 2017.

[5] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tieqiao Zheng, Bo Zhang, Junran Peng, and Qi Tian. Logformer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 135–143, 2024.

[6] P. He, J. Zhu, Z. Zheng, and M. R. Lyu. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE International Conference on Web Services (ICWS)*, pages 33–40, 2017.

[7] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R. Lyu. Lilac: Log parsing using llms with adaptive parsing cache. In *arXiv preprint arXiv:2310.01796*, 2024.

[8] V.-H. Le and H. Zhang. Log-based anomaly detection without log parsing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 492–504, 2021.

[9] Van-Hoang Le and Hongyu Zhang. Log parsing: How far can chatgpt go? In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1699–1704, 2023.

[10] Van-Hoang Le and Hongyu Zhang. Log parsing with prompt-based few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2438–2449, 2023.

[11] Yeonho Lee, Jae-Gil Kim, and Pilsung Kang. Lanobert: System log anomaly detection based on bert masked language model. *Applied Soft Computing*, 146:110689, 2023.

[12] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li. Mining invariants from console logs for system problem detection. In *USENIX Annual Technical Conference*, pages 1–14, 2010.

[13] Zeyang Ma, An Ran Chen, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. Llmparser: An exploratory study on using large language models for

log parsing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, pages 1–13, 2024.

[14] A. A. Makanju, A. N. Zincir-Heywood, and E. E. Milios. Clustering event logs using iterative partitioning. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1255–1264, 2009.

[15] W. Meng et al. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *International Joint Conference on Artificial Intelligence (IJCAI)*, volume 7, pages 4739–4745, 2019.

[16] W. Meng et al. A semantic-aware representation framework for online log analysis. In *2020 29th International Conference on Computer Communications and Networks (ICCCN)*, pages 1–7, 2020.

[17] Y. Pinter, R. Guthrie, and J. Eisenstein. Mimicking word embeddings using subword rnns. *arXiv preprint arXiv:1707.06961*, 2017.

[18] M. Schuster and K. Nakajima. Japanese and korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5149–5152, 2012.

[19] R. Vaarandi and M. Pihelgas. Logcluster—a data clustering and pattern mining algorithm for event logs. In *2015 11th International Conference on Network and Service Management (CNSM)*, pages 1–7, 2015.

[20] A. Vaswani et al. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.

[21] Xiaoyun Wu, Heng Li, and Foutse Khomh. On the effectiveness of log representation for log-based anomaly detection. In *Empirical Software Engineering*, volume 28, page 137, 2023.

[22] Zeyu Xiao et al. Demonstration-free: Towards more practical log parsing with large language models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

[23] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. Divlog: Log parsing with prompt enhanced in-context learning. In *2024 46th International Conference on Software Engineering (ICSE)*, pages 199:1–199:12, 2024.

[24] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 117–132, 2009.

[25] X. Zhang et al. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 807–817, 2019.

[26] J. Zhu et al. Tools and benchmarks for automated log parsing. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 121–130, 2019.